

Database Caching Strategies

1. Explique los desafíos que se enfrentan en la construcción de sistemas distribuidos que requieren baja latencia

- **Procesamiento lento de consultas:** Aunque existen técnicas de optimización y diseños de esquemas que mejoran el rendimiento, la velocidad de recuperación de datos desde el disco más el tiempo de procesamiento de las queries suele aumentar los tiempos de respuesta en el rango de varios milisegundos, incluso en condiciones óptimas
 - **Alto costo de escalabilidad:** Tanto en bases de datos NoSQL distribuidas en disco como en bases relacionales escaladas verticalmente, manejar una gran cantidad de lecturas resulta costoso. En muchos casos, es necesario usar múltiples réplicas de lectura para alcanzar el rendimiento que podría ofrecer un solo nodo de caché en memoria
 - **Necesidad de simplificar el acceso a los datos:** Aunque las bases de datos relacionales permiten modelar relaciones de manera eficiente, no siempre son óptimas para el acceso rápido a los datos. Algunas aplicaciones requieren estructuras o vistas específicas para facilitar la recuperación y mejorar el rendimiento, lo que añade complejidad al diseño del sistema
-

2. Explique los tipos de caching, cual considera el mas adecuado?

Existen tres tipos principales de caching en bases de datos:

Caches integrados en la base de datos:

- Se manejan directamente dentro del motor de la base de datos, actualizando el caché de forma automática cuando los datos cambian
- Su desventaja principal es que están limitados a la memoria asignada en la instancia de la base de datos y no permiten compartir información con otras instancias

Caché local:

- Almacena los datos frecuentemente usados dentro de la aplicación, eliminando tráfico de red y acelerando el acceso
- Sin embargo, cada nodo mantiene su propio caché aislado, lo que dificulta la sincronización en entornos distribuidos. Además, la información se pierde en caso de caídas, lo que obliga a regenerar el caché

Caché remoto:

- Se implementa en servidores dedicados e independientes, generalmente con tecnologías como Redis o Memcached
- Ofrece altísimo rendimiento (cientos de miles o millones de solicitudes por segundo) y latencias de sub-milisegundos, siendo mucho más rápido que una base de datos en disco

- Es ideal para entornos distribuidos porque permite a múltiples sistemas compartir el caché como un clúster conectado, garantizando además alta disponibilidad

Por lo tanto, el caché remoto es el más adecuado, especialmente en sistemas distribuidos, porque combina velocidad, escalabilidad, alta disponibilidad y la capacidad de ser compartido entre diferentes aplicaciones, superando las limitaciones de los otros enfoques

3. Explique las diferencias entre los patrones de caching

Los dos patrones más comunes son cache-aside y write-through, y se diferencian principalmente en cuándo y cómo se actualiza la caché:

Cache-aside

- La aplicación consulta primero la caché. Si no encuentra los datos, los obtiene de la base de datos y luego los guarda en la caché
- Solo se almacenan datos realmente solicitados, lo que mantiene la caché eficiente y económica
- Su implementación es sencilla y ofrece mejoras inmediatas en el rendimiento
- En el primer acceso a un dato que no está en la caché se suele demorar, ya que se requiere consultar la caché y luego a la base de datos

Write-through

- Cada vez que se actualiza la base de datos, los datos se escriben también en la caché de inmediato
- La caché siempre está actualizada, lo que hace más probable que la información solicitada esté en la caché
- Se reducen las lecturas directas a la base de datos, optimizando su rendimiento
- Se almacenan datos que quizás nunca se consulten, lo que puede hacer la caché más grande y costosa

Entonces la principal diferencia es que el patrón cache-aside carga los datos solo cuando se necesitan, mientras que el write-through mantiene los datos siempre actualizados

4. Explique que es cache eviction, por que es relevante para el rendimiento de un sistema?

Cache eviction ocurre cuando la memoria de la caché se llena y el sistema necesita liberar espacio eliminando ciertas claves. Esto se hace según una política de expulsión que define qué datos se eliminan primero

Las políticas de expulsión son:

- **LRU:** elimina los datos menos usados recientemente
- **LFU:** elimina los datos menos accedidos
- **TTL-based:** elimina los datos con menor tiempo de vida restante
- **Random:** elimina claves al azar
- **No eviction:** bloquea nuevas escrituras si no hay espacio

Es relevante porque en la forma en que se gestionan las expulsiones afecta directamente el rendimiento del sistema. Si se eliminan datos útiles con frecuencia, se reduce la cantidad de aciertos en la caché, lo que obliga a hacer más lecturas desde la base de datos y aumenta la latencia